

RENTAL MANAGEMENT SYSTEM

Backend API Development Report



Submitted By:

Tanishq Dhote
Prithviraj Kshirsagar
Sudesh Jambhulkar

Date: June 18, 2026

Table of Contents

1. Project Overview
2. Technology Stack & Tools Used
3. Database Schema Design (ER)
4. Application Architecture
5. Postman Integration & Testing Guide

1. Project Overview

The Rental Management System is a web service designed to facilitate tenancy leases, property listings, and secure user management. Built with FastAPI and backed by a relational PostgreSQL database, it provides robust APIs that allow landlords to list rental properties and tenants to search for and lease available accommodations.

Security is managed using industry-standard JSON Web Tokens (JWT) for authentication. Additionally, role-based authorization enforces access control, ensuring only authorized landlord accounts can modify property listings while tenant accounts are permitted to sign tenancy contracts.

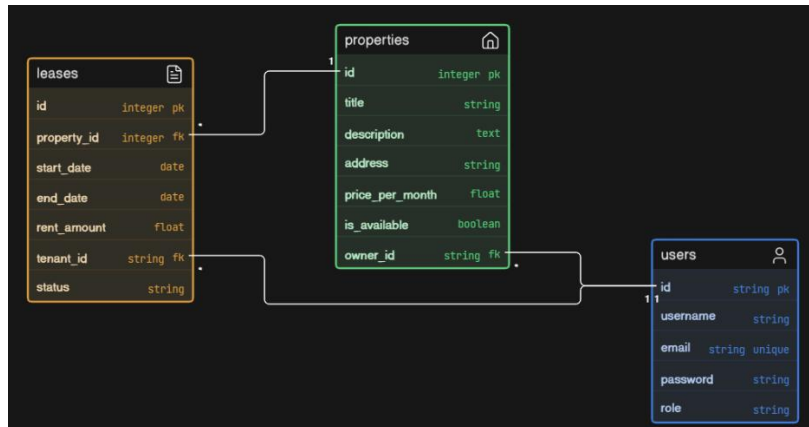
2. Technology Stack & Tools Used

The following tools and frameworks were utilized to build and run the application:

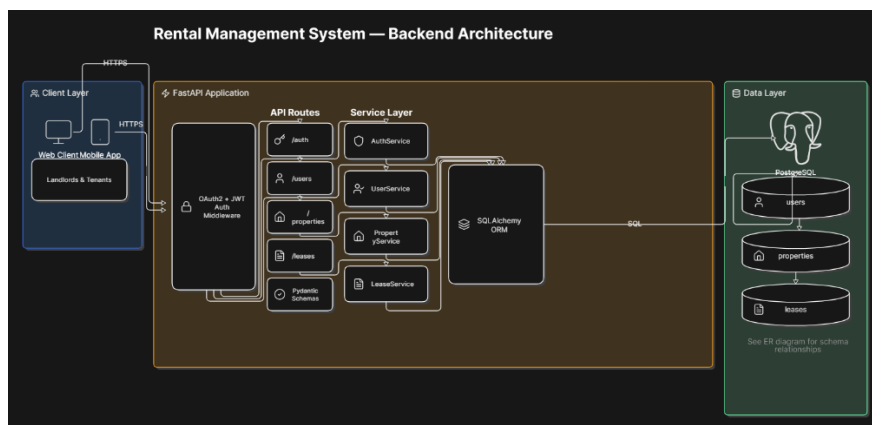
Technology / Tool	Purpose & Role in Project
FastAPI	Python web framework used for designing the RESTful endpoints, validating requests, and generating automatic Swagger documentation.
PostgreSQL	Relational Database Management System (RDBMS) used for secure, relational, and persistent data storage.
SQLAlchemy	Object-Relational Mapping (ORM) library used to interact with the PostgreSQL database using Python classes (models) instead of raw SQL queries.
Uvicorn	Asynchronous Server Gateway Interface (ASGI) web server used to run the FastAPI application locally.
Pydantic	Data parsing and validation library used to define request and response payloads (schemas) with strict type checking.
python-jose & bcrypt	Libraries used to securely hash user passwords (using bcrypt) and generate/validate signed JSON Web Tokens (JWT).
Postman	API client used for manually calling, testing, and verifying the routes during development.

3. Database Schema Design (ER)

The relational database schema is structured into three main tables: **users**, **properties**, and **leases**. Below is the relational structure of the tables:



4. Architecture



4.1 Request Flow and Business Rules

- **Authentication Check:** Endpoint paths that require authentication use the `get_current_user` dependency, which decodes the JWT token from the HTTP Bearer header, validates it against the signing key, and retrieves the matching user record from the database.
- **Role Restriction:** Landlord endpoints (e.g. creating/updating properties) are protected by `check_landlord`, which returns a 403 Forbidden error if a tenant tries to list a house.
- **Lease Automation:** When a tenant successfully leases a property (`POST /leases`), the system automatically registers the lease in the database and updates the property's `is_available` column to `False`. When a lease is terminated (`POST /leases/{id}/terminate`), the property availability is set back to `True`.

5. Postman Integration & Testing Guide

5.1 Sign Up (`POST /signup`)

POST /signup Create new user

Parameters

CancelReset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "username": "Kalpesh Kumar",
  "email": "kk@gmail.com",
  "password": "kk12345",
  "role": "tenant"
}
```

ExecuteClear

Responses

200

curl -X 'POST' \
'http://127.0.0.1:8000/signup' \
-H 'accept: application/json' \
-H 'Content-type: application/json' \
-d '{
 "username": "Kalpesh Kumar",
 "email": "kk@gmail.com",
 "password": "kk12345",
 "role": "tenant"
}'

Request URL

5.2 Login & Token Retrieval (`POST /login`)

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1eHA0IjE3ODU3NTg4MDkzInN1Y1I6InZpbmF5c08nb29nbGUyZ3R1In0.Nz99P9pi-rmZSCJYdI-E19PaStGUGwVBF-2ydyf=7YA",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1eHA0IjE3ODU3NTg4MDkzInN1Y1I6InZpbmF5c08nb29nbGUyZ3R1In0.guXfYParj8JLfnBDJ5zn80R7NN8en_EyUuHuYseo18",
  "token_type": "bearer"
}
```

Download

Response headers

```
content-length: 340
content-type: application/json
date: Thu, 18 Jun 2020 04:38:09 GMT
server: uvicorn
```

Responses

CodeDescriptionLinks

5.3 Calling Authenticated Routes (e.g., `POST /properties`)

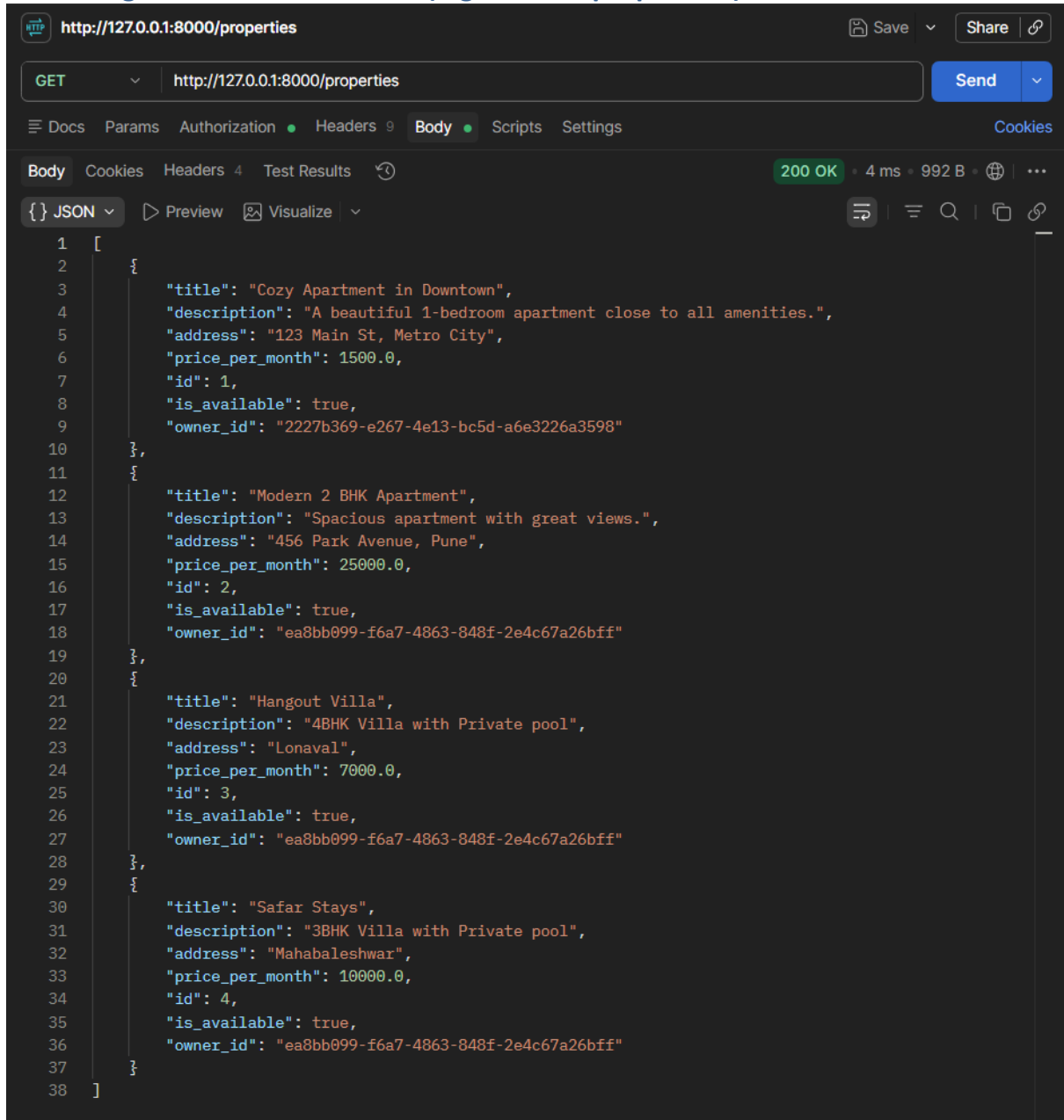
The screenshot displays an HTTP client interface with the following components:

- URL Bar:** Shows the URL `http://127.0.0.1:8000/properties` with a `Save` button and a `Share` link.
- Request Configuration:** The `POST` method is selected, and the `Send` button is visible.
- Request Body:** The `Body` tab is active, showing a JSON payload in raw format:

```
1 {  
2   "title": "Safar Stays",  
3   "description": "3BHK Villa with Private pool",  
4   "address": "Mahabaleshwar",  
5   "price_per_month": 10000  
6 }
```
- Response Status:** The status bar indicates a `200 OK` response with a duration of `112 ms` and a size of `323 B`.
- Response Body:** The `JSON` tab is active, showing the response payload in a formatted view:

```
1 {  
2   "title": "Safar Stays",  
3   "description": "3BHK Villa with Private pool",  
4   "address": "Mahabaleshwar",  
5   "price_per_month": 10000.0,  
6   "id": 4,  
7   "is_available": true,  
8   "owner_id": "ea8bb099-f6a7-4863-848f-2e4c67a26bff"  
9 }
```

5.4 Calling Authenticated Routes (e.g., `POST /properties`)



The screenshot shows a web browser's developer tools interface. The address bar displays `http://127.0.0.1:8000/properties`. The network tab is active, showing a `GET` request to the same URL. The response status is `200 OK` with a response time of `4 ms` and a size of `992 B`. The response body is displayed in JSON format, showing an array of four property objects. The JSON is as follows:

```
[
  {
    "title": "Cozy Apartment in Downtown",
    "description": "A beautiful 1-bedroom apartment close to all amenities.",
    "address": "123 Main St, Metro City",
    "price_per_month": 1500.0,
    "id": 1,
    "is_available": true,
    "owner_id": "2227b369-e267-4e13-bc5d-a6e3226a3598"
  },
  {
    "title": "Modern 2 BHK Apartment",
    "description": "Spacious apartment with great views.",
    "address": "456 Park Avenue, Pune",
    "price_per_month": 2500.0,
    "id": 2,
    "is_available": true,
    "owner_id": "ea8bb099-f6a7-4863-848f-2e4c67a26bff"
  },
  {
    "title": "Hangout Villa",
    "description": "4BHK Villa with Private pool",
    "address": "Lonaval",
    "price_per_month": 7000.0,
    "id": 3,
    "is_available": true,
    "owner_id": "ea8bb099-f6a7-4863-848f-2e4c67a26bff"
  },
  {
    "title": "Safar Stays",
    "description": "3BHK Villa with Private pool",
    "address": "Mahabaleshwar",
    "price_per_month": 10000.0,
    "id": 4,
    "is_available": true,
    "owner_id": "ea8bb099-f6a7-4863-848f-2e4c67a26bff"
  }
]
```